ANÁLISE DE ALGORITMOS

Matheus da Silva Serpa



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

Modelos de computação e eficiência de algoritmos

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Definir complexidade temporal e complexidade de espaço.
- Desenvolver um modelo de contagem para análise da eficiência de algoritmos.
- Explicar a contagem na análise de eficiência de algoritmos de melhor e pior caso.

Introdução

Entender o funcionamento e a eficiência de algoritmos é fundamental para a resolução de problemas de ciência de dados e de outras diversas áreas da computação e afins. O uso da estrutura de dados e da solução computacionalmente mais eficiente pode alterar o tempo de uma simulação ou a velocidade de um aplicativo em dezenas ou, até mesmo, centenas de vezes.

Neste capítulo, você vai entender melhor os conceitos de complexidade temporal e de espaço ao aprender como avaliar o consumo de tempo e de memória utilizado por um algoritmo. Além disso, vamos desenvolver um modelo de contagem para analisar a eficiência de algoritmos sem levar em consideração todos os detalhes do dispositivo no qual o algoritmo está executando. Por fim, você vai ver que o desempenho de um algoritmo pode variar de acordo com a entrada utilizada e, devido a isso, análises de eficiência são feitas para o pior e o melhor caso de entrada.

1 Complexidade de tempo e espaço

O processo de análise de algoritmos tem envolvimento direto com os requisitos do programa, os quais podem variar de acordo com seus parâmetros de entrada. Na maior parte das vezes, estamos interessados em dois requisitos: a quantidade de tempo e a quantidade de memória que um algoritmo ou programa utiliza em relação ao tamanho de sua entrada. Essas duas medidas são chamadas de complexidade de tempo e complexidade de espaço, respectivamente.

Conforme Kleinberg e Tardos (2006), um algoritmo é eficiente se, quando implementado, é executado rapidamente em entradas de casos reais. O ponto mais importante aqui é entender alguns aspectos cruciais que não são levados em consideração nessa definição. Vamos assumir a existência de um algoritmo que não foi bem implementado. Esse algoritmo pode ser executado rapidamente em um processador moderno quando testado com um caso de teste pequeno. Entretanto, quando uma entrada maior for utilizada, o desempenho pode mudar completamente.

Outro aspecto importante é qual seria a definição de **entrada real**. Existem diversos problemas cujas entradas que serão executadas na prática não conhecemos, sendo que um conjunto de entradas pode ser muito mais difícil que outro conjunto. Por exemplo, um algoritmo que busca um elemento em um vetor percorrendo todos N elementos dele. Se o elemento buscado não estiver no vetor, vamos ter que olhar todas as posições, e isso vai ser mais lento do que uma entrada na qual o elemento é o primeiro do vetor, o qual seria identificado na primeira comparação.

Além disso, esse tipo de definição não considera o que acontece quando aumentamos a entrada em 10, 100 vezes. Podem existir dois algoritmos, os quais, para uma entrada de tamanho 100, têm o mesmo desempenho, mas, para uma entrada 10 vezes maior, um deles continuará muito rápido e o outro executará de forma muito lenta. Portanto, o que buscamos é uma definição concreta de **eficiência**, que será independente do processador, da plataforma e da instância que estamos executando.



Saiba mais

O tamanho da instância do problema analisado é muito importante para análise de um algoritmo. Digite "Problemas, instâncias, algoritmos, tempo" em seu motor de busca preferido para ter acesso a uma aula do prof. Paulo Feofiloff (Universidade de São Paulo) sobre o assunto (FEOFILOFF, 2020).

Ao analisar um algoritmo, em geral vamos considerar as complexidades de tempo e de espaço. A **complexidade de tempo** indica o tempo que um algoritmo leva para executar de acordo com sua entrada. De forma similar, a **complexidade de espaço** diz respeito à quantidade de espaço em memória ocupado por um algoritmo também em relação ao tamanho de sua entrada.

Ambas as complexidades dependem de diversos aspectos de *hardware* e *software*, como a velocidade do processador, a frequência das memórias, o compilador e o sistema operacional, por exemplo. No entanto, quando analisarmos a complexidade de algoritmos, não vamos levar esses fatores em consideração.

Vamos começar com um exemplo de algoritmo que soma vetores. Mas, antes disso, é importante mencionar que, a partir de agora, em todos os pseudocódigos, assumiremos que os vetores possuem índices que vão de 1 até N e que o laço `para` é fechado à direita, ou seja, inclui o valor N. O comportamento das implementações em Python difere um pouco disso, pois, nessa linguagem, os índices dos vetores vão de 0 até N – 1. Além disso, o laço `for` é aberto à direita, ou seja, não inclui o valor N.

Pseudocódigo	Implementação em Python

`SOMA-VETORES[A, B, N]`
`1. para i = 1 até N`
`2.  C[i] = A[i] + B[i]`
`3. retorna C`
	`def soma_vetores(A, B, N):`
	`  for i in range (0, N):`
	`  C[i] = A[i] + B[i]`
	`  return C`

Nesse algoritmo, temos, na linha 1, um laço de repetição que percorre os N elementos do vetor. Na complexidade de tempo, dizemos que esse laço tem custo linear, ou seja, custo N. As operações na linha 2 são alinhadas ao laço `para` da linha 1; ou seja, em cada iteração, elas serão executadas uma vez. Visto que o número de operações da linha 2, em cada iteração, não cresce com o aumento do valor N e que o custo dessas operações é menor que o custo das N iterações, dizemos que a linha 2 tem um custo constante. Dessa maneira, esse algoritmo tem uma complexidade de tempo N. Já a complexidade de espaço desse algoritmo se refere à quantidade de memória extra que não diz respeito à entrada e à saída do algoritmo. Nesse caso, não existe uso de memória extra, então a complexidade de espaço é constante, ou seja, 1.



Fique atento

Quando estamos analisando a complexidade de tempo de um algoritmo, levamos em consideração, principalmente, os laços dele, pois as operações mais custosas estão sendo representadas neles. Também é importante que esses laços avaliados tenham uma quantidade de iterações que se relacionem com o tamanho da entrada.

Um segundo exemplo interessante de analisar mostra a função `merge` do algoritmo de ordenação *merge sort*. Essa função possui diversos laços e utiliza memória extra, o que deixa claras as diferenças entre a complexidade de tempo e a de espaço.

O *merge sort* é um algoritmo de ordenação implementado utilizando a técnica de divisão e conquista, sendo que, durante sua execução, a função `merge` é chamada diversas vezes. A função `merge` tem como objetivo combinar dois subvetores já ordenados, resultando em um vetor ordenado. Para tanto, dois ponteiros (`i` e `j`) são mantidos para comparar os dois subvetores, encontrando as posições corretas no vetor original.

Pseudocódigo

```
MERGE[A, esq, meio, dir]
   V[dir – esq + 1] = []
   i = esq
   j = meio + 1
   k = 1
   enquanto i <= meio e j <= dir
      se A[i] < A[j] então
         V[k] = A[i]
         i = i + 1
      senão
         V[k] = A[j]
         j = j + 1
      k = k + 1
   enquanto i <= meio
      V[k] = A[i]
      i = i + 1
      k = k + 1
   enquanto j <= dir
      V[k] = A[j]
      j = j + 1
      k = k + 1
   para i = 1 até k
      A[esq + i] = V[i]
```

Implementação em Python

```
1. def merge(lista, esq, meio, dir):
2.     v = [None] * (dir - esq + 1)
3.     i = esq
4.     j = meio + 1
5.     k = 0
6.
7.     while(i <= meio and j <= dir):
8.         if(lista[i] < lista[j]):
9.             v[k] = lista[i]
10.            i = i + 1
11.        else:
12.            v[k] = lista[j]
13.            j = j + 1
14.        k = k + 1
15.
16.    while(i <= meio):
17.        v[k] = lista[i]
18.        i = i + 1
19.        k = k + 1
20.
21.    while(j <= dir):
22.        v[k] = lista[j]
23.        j = j + 1
24.        k = k + 1
25.
26.    for i in range (0, k):
27.        lista[esq + i] = v[i]
```

No pseudocódigo da função `merge`, temos, como entrada, um vetor A, o qual possui diversos subvetores ordenados, que precisam ser combinados com objetivo de possuir todo o vetor ordenado. Em uma chamada da função `merge`, o objetivo é combinar dois subvetores ordenados em um vetor ordenado. A variável `esq` indica a posição inicial do subvetor1 e a `meio` indica a posição final do subvetor1. Assim, `meio + 1` é o início do subvetor2 e `dir` é a posição final desse vetor.

Analisando a complexidade, na linha 1, podemos ver uma alocação de memória extra, a qual não tem relação com a entrada do algoritmo. Esse vetor alocado possui (`dir - esq + 1`) posições, alocadas para armazenar o vetor ordenado após a intercalação. Esse vetor, na última chamada da função `merge`, tem tamanho de N posições. Isso resulta em uma complexidade de espaço de N para a função `merge`.

No que diz respeito à complexidade de tempo, primeiramente temos que identificar os laços da função. Eles podem ser encontrados nas linhas 5, 13, 17 e 21. Nenhum dos laços encontra-se alinhado; logo, o desempenho do algoritmo será indicado pelo laço mais custoso. O laço da linha 5 possui complexidade N, pois, no pior caso, esse é o valor máximo que as variáveis `meio + dir` podem alcançar. O mesmo pode ocorrer nas linhas 13 e 17; porém, devido à característica do algoritmo, quando um dos laços executa, o outro não executará, de forma que teremos uma complexidade N apenas uma vez para esses dois laços. Por fim, o laço da linha 21 também executa N vezes; ou seja, a complexidade de tempo desse algoritmo é linear, isto é, N.

2 Modelo de contagem simplificado

Como mencionado anteriormente, quando analisamos a complexidade de um algoritmo, não levamos em consideração aspectos de *hardware* e *software* como o modelo do processador, sua frequência de *clock*, o sistema operacional e o compilador. Nosso objetivo não é definir com extrema precisão o desempenho de um algoritmo em um computador ou servidor específico: o que desejamos é um modelo simplificado com o qual podemos comparar o desempenho de diferentes soluções para problemas computacionais quando executando entradas grandes.

Como um primeiro exemplo, vamos utilizar o algoritmo de ordenação *insertion sort*, o qual ordena uma lista de valores de acordo com determinada ordem, como do menor para o maior ou em ordem alfabética, por exemplo. A prova da corretude desse algoritmo pode ser verificada em Cormen *et al.*

(2012). No Quadro 1, assumimos que a entrada do algoritmo é um vetor A de N posições indexado de 1 até N. No mesmo quadro, vamos ver a análise com um modelo de contagem detalhado e um simplificado, ficando claro o motivo pelo qual decidimos utilizar um modelo simples e menos complexo. No caso do modelo detalhado, para cada linha do código, teremos um custo Ci e uma quantidade de execuções calculada de forma precisa. Já para o modelo simplificado, teremos o número de execuções apenas para os laços; para as outras linhas, vamos utilizar o valor constante, representado pelo número 1.

Quadro 1. Análise da complexidade de tempo do algoritmo *insertion sort*

#	Pseudocódigo	Modelo detalhado		Modelo simplificado
		Custo	Execuções	
1	**para** `j = 2` **até** `N`	C1	N	N
2	`chave = A[j]`	C2	N – 1	1
3	`i = j - 1`	C3	N – 1	1
4	**enquanto** `i > 0 e A[i] > chave`	C4	$\sum_{j=2}^{N} t_j$	N
5	`A[i + 1] = A[i]`	C5	$\sum_{j=2}^{N}(t_j - 1)$	1
6	`i = i - 1`	C6	$\sum_{j=2}^{N}(t_j - 1)$	1
7	`A[i + 1] = escolhido`	C7	N – 1	1

No caso do modelo detalhado, a complexidade de tempo total desse algoritmo de ordenação é a soma da multiplicação entre os custos e a quantidade de execuções. Após resolver os somatórios e realizar as multiplicações, teríamos a equação a seguir:

$$\left(\frac{c_5}{2}+\frac{c_6}{2}+\frac{c_7}{2}\right)N^2+\left(c_1+c_2+c_4+\frac{c_5}{2}-\frac{c_6}{2}-\frac{c_7}{2}+c_8\right)N-(c_2+c_4+c_5+c_8)$$

Essa equação também pode ser expressa por $aN^2 + bN^2 + c$, onde a, b e c são constantes. O problema é que esse tipo de modelo detalhado dificulta a análise da complexidade de tempo dos algoritmos, e quando grandes tamanhos de entrada são utilizados, essa abordagem não adiciona uma precisão tão maior do que outros modelos de contagem simples, como os que só levam em consideração as operações e os termos mais custosos das funções. Utilizando um modelo de contagem simples, apenas os laços das linhas 1 e 4 são considerados, pois esses são os mais custosos, cada um tendo um tempo de execução igual a N, os quais, sendo dois laços que estão alinhados, resultam em uma complexidade de tempo N * N, ou seja, N^2.

Continuando, podemos tomar como exemplo um caso em que, ao analisar a complexidade de tempo de determinado algoritmo para entradas de tamanho N, poderíamos detalhadamente indicar que seu desempenho é de $1{,}6N^2 + 3{,}4N + 7$. Por outro lado, de forma simplista, poderíamos indicar que sua complexidade de tempo é N^2. Executando esse algoritmo com uma entrada de tamanho N = 1.000.000, a análise detalhada resultaria em 1,6 trilhões, enquanto, utilizando a forma simplista, resultaria em 1 trilhão.

Em alguns contextos, pode ser que a opção de análise detalhada seja relevante, mas, em geral, desejamos comparar algoritmos quando executando entradas muito grandes, no intuito de identificar algoritmos que possuem comportamentos melhores, piores e parecidos. Obter uma complexidade tão precisa é uma tarefa exaustiva, sendo que nosso objetivo é identificar classes amplas de algoritmos com comportamento semelhante.



Saiba mais

Leia mais sobre algoritmos e suas complexidades de tempo e espaço na obra *Algoritmos*, de Dasgupta, Papadimitriou e Vazirani (2009).

No Quadro 2, podemos ver as principais complexidades de tempo de algoritmos e seus respectivos tempos de execução. Além disso, o tamanho de entrada é variado, mostrando que, para valores de N pequenos, a complexi-

dade de alguns algoritmos não afeta a eficiência, mas que, para outros casos, como aqueles em que o valor de N é grande, a complexidade é relevante. Os dados apresentados no Quadro 2 assumem a execução dos algoritmos em um computador com processador de frequência 1 GHz.

Quadro 2. Principais complexidades de tempo de algoritmos e seus respectivos tempos de execução

Entrada	N	N log N	N^2	N^3	2^N	N!
10	< 1 seg	< 1 seg	< 1 seg	< 1 seg	< 1 seg	4 seg
30	< 1 seg	< 1 seg	< 1 seg	< 1 seg	18 min	10^{25} anos
50	< 1 seg	< 1 seg	< 1 seg	< 1 seg	36 anos	Muito tempo
100	< 1 seg	< 1 seg	< 1 seg	1 seg	10^{17} anos	Muito tempo
1 000	< 1 seg	< 1 seg	1 seg	18 min	Muito tempo	Muito tempo
10 000	< 1 seg	< 1 seg	2 min	12 dias	Muito tempo	Muito tempo
100 000	< 1 seg	2 seg	3 horas	32 anos	Muito tempo	Muito tempo
1 000 000	1 seg	20 seg	12 dias	31 710 anos	Muito tempo	Muito tempo

Fonte: Adaptado de Kleinberg e Tardos (2006).

Para entradas pequenas até 10.000, é possível ver que algoritmos com complexidades N e N log N possuem desempenho parecido, ambos menores que 1 seg. Já quando o tamanho de entrada é 1 milhão, por exemplo, o tempo de um é 1 segundo e do outro é 20 segundos. Quando analisamos algoritmos com complexidades N^2 e N^3, as diferenças são maiores. Para uma entrada de tamanho 1.000, o algoritmo N^2 executa em 1 segundo, enquanto o N^3 executa em 18 min. Por fim, quando algoritmos 2^N e N!, menos eficientes, são apresentados, para uma entrada de tamanho 50, o 2^N leva 36 anos e o N! leva uma quantidade inviável de anos.

3 Complexidade de pior e de melhor caso

Ao analisar a complexidade de tempo e de espaço de algoritmos, não analisamos todas entradas possíveis para o algoritmo, as quais podem, dependendo do caso, alterar sua eficiência e complexidade. Logo, incialmente, vamos nos concentrar em analisar o pior caso do algoritmo; ou seja, vamos analisar o algoritmo assumindo que a entrada utilizada é a pior possível para aquele algoritmo. Isso vai fornecer um limite superior, o qual vai indicar o maior tempo de execução possível que o algoritmo possa ter para todas as entradas de um tamanho N, por exemplo.

Focar nossa análise apenas no pior caso seria excessivamente rigoroso, pois pode acontecer de um algoritmo ter um bom desempenho na maior parte das instâncias e apenas em poucos casos atingir o desempenho de pior caso. Um exemplo de algoritmo que possui esse comportamento é a ordenação *quick sort*, cuja análise de complexidade pode ser verificada em Toscani e Veloso (2012). Isso é um problema em alguns casos, mas, em geral, veremos que a análise de pior caso de um algoritmo é realmente capaz de capturar a eficiência dos algoritmos na prática. Além do pior caso, é comum analisar o melhor caso, sendo que esse indica o limite inferior. Resumidamente, com essas duas análises, podemos indicar o comportamento geral do algoritmo e das aplicações.

Para entender melhor as análises de melhor e de pior caso de algoritmos, vamos resolver o problema de buscar um elemento em uma lista de valores. Para esse problema, uma das soluções é a implementação de uma busca sequencial, a qual percorre toda a lista, da esquerda para a direita, buscando pelo elemento desejado. No algoritmo de busca sequencial, os três parâmetros são a lista, o valor buscado e um valor N, que indica o número de elementos da lista. Após, utilizando um laço, a lista é percorrida do início ao fim. Em cada iteração, é feito um teste que verifica se o valor atual da lista é o valor buscado. Caso isso seja verdadeiro, o índice do elemento é retornado; caso contrário, a execução continua até a lista acabar. Nesse caso, uma mensagem "não encontrado" é retornada ou, no caso da implementação em Python, o valor `None`.

Pseudocódigo	Implementação em Python

BUSCA-SEQUENCIAL[LISTA, VALOR, N]
1. **para** i = 1 **até** N
2. **se** LISTA[i] == VALOR **então**
3. **retorna** i
4. **retorna** "não encontrado"
	def busca_sequencial(L, v, N):
	for i in range (0, N):
	if L[i] == v:
	return i
	return None

Dada a lista a seguir com N = 8 valores, vamos procurar o caso que resultaria no pior desempenho e o que resultaria no melhor desempenho para esse algoritmo de busca sequencial.

4	2	8	1	10	6	7	0

O melhor caso de execução ocorre quando o elemento buscado é o primeiro da lista, ou seja, o primeiro a ser verificado. Nesse caso, independentemente de se o tamanho da lista é N = 8 ou N = 1 bilhão, apenas uma iteração do algoritmo é executada. Assim, dizemos que a complexidade de tempo de melhor caso da busca sequencial é de ordem constante 1. Por outro lado, o pior caso de execução ocorre quando o elemento buscado não está contido na lista ou é o último da lista, pois, nesse caso, toda a lista é percorrida e o elemento não é encontrado. Dessa maneira, independentemente do tamanho da lista, o número de iterações do laço será N, resultando em uma complexidade de tempo de pior caso de ordem linear N.

Portanto, podemos dizer que a busca sequencial possui complexidade de tempo constante no melhor caso e linear no pior caso. No que diz respeito à complexidade de espaço, precisamos analisar quanto de espaço extra, que não as entradas e saídas, que as chamadas da função utilizam. No caso da implementação da busca sequencial, o espaço extra utilizado é apenas o da variável de controle do laço, a variável `i`; sendo essa apenas uma variável e seu tamanho não aumentando com o incremento do tamanho da entrada, dizemos que a complexidade de espaço desse algoritmo é constante também.

Um segundo problema que podemos utilizar para entender melhor as complexidades de tempo de melhor e de pior caso de algoritmos é o problema de calcular um termo específico de uma progressão geométrica (PG). Uma PG é uma sequência numérica que possui uma razão (R) e um termo inicial (A). O termo (N), pode ser calculado a partir dos cálculos da soma da razão multiplicada pelo antecessor.

Pseudocódigo	Implementação em Python

PG-ITERATIVO[A, R, N]
1. soma = A
2. **para** i = 1 **até** N
3. soma = soma * R
4. **retorna** soma
	`def pg_iterativo(A, R, N):`
	`  soma = A`
	`  for i in range(1, N):`
	`  soma = soma * R`
	`  return soma`

No pseudocódigo e na implementação em Python acima, podemos ver uma proposta de solução para o problema de encontrar o N-ésimo valor de um PG. Essa é uma solução iterativa; ou seja, calculamos o valor da posição N percorrendo um laço e fazendo as somas. Na linha 2, podemos ver o laço que possui complexidade de tempo linear N. Nesse algoritmo, independentemente da entrada, tanto o pior quanto o melhor caso terão a mesma complexidade de tempo, pois o valor da entrada não afeta o número de iterações desse laço. Essa solução para o problema, mesmo que simples, ao ser inserida dentro de uma aplicação maior, afetaria seu desempenho. Nesse sentido, sempre buscamos propor soluções mais eficientes.

Pseudocódigo	Implementação em Python
PG-FORMULA[A, R, N] 1. soma = $A * R^{N-1}$ 2. **retorna** soma	def pg_iterativo(A, R, N): soma = A * R ** (N - 1) return soma

Uma segunda solução para esse problema é mostrada acima; nela, em vez de um laço iterativo, usamos uma forma fechada para chegar ao resultado. Nesse caso, novamente, em ambos, melhor caso e pior caso de sua execução, o desempenho será o mesmo. Uma vez que não existem laços de repetição, a complexidade de tempo de melhor e pior caso desse algoritmo é constante, ou seja, O (1). A complexidade de espaço também é constante, uma vez que não utilizamos vetores extras.

Neste capítulo, estudamos as complexidades de tempo e de espaço, utilizadas para medir a eficiência de um algoritmo na solução de um problema. Além disso, desenvolvemos um modelo de contagem simples, mas eficiente, o qual abstrai aspectos de *hardware* e *software*. Por fim, estudamos as complexidades de melhor e de pior caso, as quais indicam o melhor e o pior desempenho que o algoritmo tende a alcançar.

Referências

CORMEN, T. H. *et al. Algoritmos*: teoria e prática. 3. ed. Rio de Janeiro: Elsevier, 2012.

DASGUPTA, S.; PAPADIMITRIOU, C.; VAZIRANI, U. *Algoritmos*. Porto Alegre: AMGH, 2009.

FEOFILOFF, P. Minicurso de análise de algoritmos. *In:* INSTITUTO de Matemática e Estatística da USP. São Paulo, 2 ago. 2020. Disponível em: https://www.ime.usp.br/~pf/livrinho-AA/AA-BOOKLET.pdf. Acesso em: 4 set. 2020.

KLEINBERG, J.; TARDOS, E. *Algorithm design*. Harlow: Pearson Education, 2006.

TOSCANI, L. V.; VELOSO, P. A. *Complexidade de algoritmos*. 3. ed. Porto Alegre: Bookman, 2012. (Série Livros Didáticos Informática UFRGS)



Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS